1. <u>Text Cleaning Pipeline</u>

We applied a standard NLP text cleaning pipeline to prepare raw text for downstream tasks. The following steps were performed:

- **Lowercasing**
  Converts all characters to lowercase to ensure uniformity.
  Example: "This is TEXT" → "this is text"

- **Removing Punctuation**
  Strips punctuation marks such as .,!?;: using regex or tokenizer settings.
  Example: "hello, world!" → "hello world"

- **Removing Stopwords**
  Commonly used words like "the", "is", "and" were removed to reduce noise.
  Example: "this is a test" → "test"

- **Removing Numbers**
  Digits were removed as they often do not contribute semantic value (depends on use case).
  Example: "I have 2 dogs" → "I have dogs"

- **Tokenization**
  The cleaned text was split into individual words (tokens).
  Example: "I love NLP" → ["I", "love", "NLP"]

- **Stemming / Lemmatization**

Pipeline Overview

Raw Text → Lowercase → Remove Punctuation → Remove Stopwords → Remove Numbers → Tokenize → Stem / Lemmatize

2. <u>Stemming vs Lemmatization</u>

Stemming and lemmatization are techniques used to reduce words to their base or root form, but they differ in method and accuracy.

Comparison Table

Feature	**Stemming**	**Lemmatization**
Description	Truncates suffixes crudely	Uses dictionary and POS tagging
Output	May not be actual word	Returns valid dictionary word
Speed	Fast	Slower
Accuracy	Low (may over-stem)	High (context-aware)
Tool Used	PorterStemmer (NLTK)	WordNetLemmatizer (NLTK)

Example Transformations

Original Text (truncated)	**Stemmed Output (truncated)**	**Lemmatized Output (truncated)**
"Figure people enjoyed unbelievable predictable..."	"figur peopl enjoy unbeliev predict werent realli film like..."	"figure people enjoyed unbelievable predictable weren't really film like..."
"Really enjoyed account sinking Titanic..."	"realli enjoy account sink titan heart realli well done..."	"really enjoyed account sinking titanic heart really well done..."
"Second trilogy launched one important fantasy writer..."	"second trilog launch one import fantasi writer time..."	"second trilogy launched one important fantasy writer time..."
"History books given rather wide view tapestry..."	"histori book given rather wide view tapestri interest topic..."	"history book given rather wide view tapestry interest topic..."
"La verdad es que interesaba hablar de encarnación..." (Spanish, untranslated)	Unchanged — non-English text	Slight change — partial lemmatization (e instead of es)

Observations from Sample

- **Stemming** significantly reduces tokens, often removing meaningful suffixes: *"fantasy"* → *"fantasi"*, *"trilogy"* → *"trilog"*, *"sinking"* → *"sink"*
- **Lemmatization** keeps syntactic and semantic integrity: *"enjoyed"* → *"enjoyed"*, *"launched"* → *"launched"*
- **Stemming** sometimes produces non-words (e.g., *"tapestri"*, *"histori"*), while lemmatization usually produces dictionary-valid words.
- **Non-English text** (e.g., Spanish) may remain unchanged or partially modified if language-specific models are not used.

Given that reviews contain nuances that can be lost with aggressive stemming, **lemmatization** seems to be the **better choice** for this task. It better preserves the context and meaning of words, which is crucial when working with sentiment analysis and content-based analysis.

3. Token Statistics

After cleaning and preprocessing a sample corpus, the following token statistics were observed:

Metric	**Value**
Tokens before cleaning	19405344
Tokens after cleaning	19316314
Unique tokens after cleaning	421367
Average tokens per document	257.86372798996115

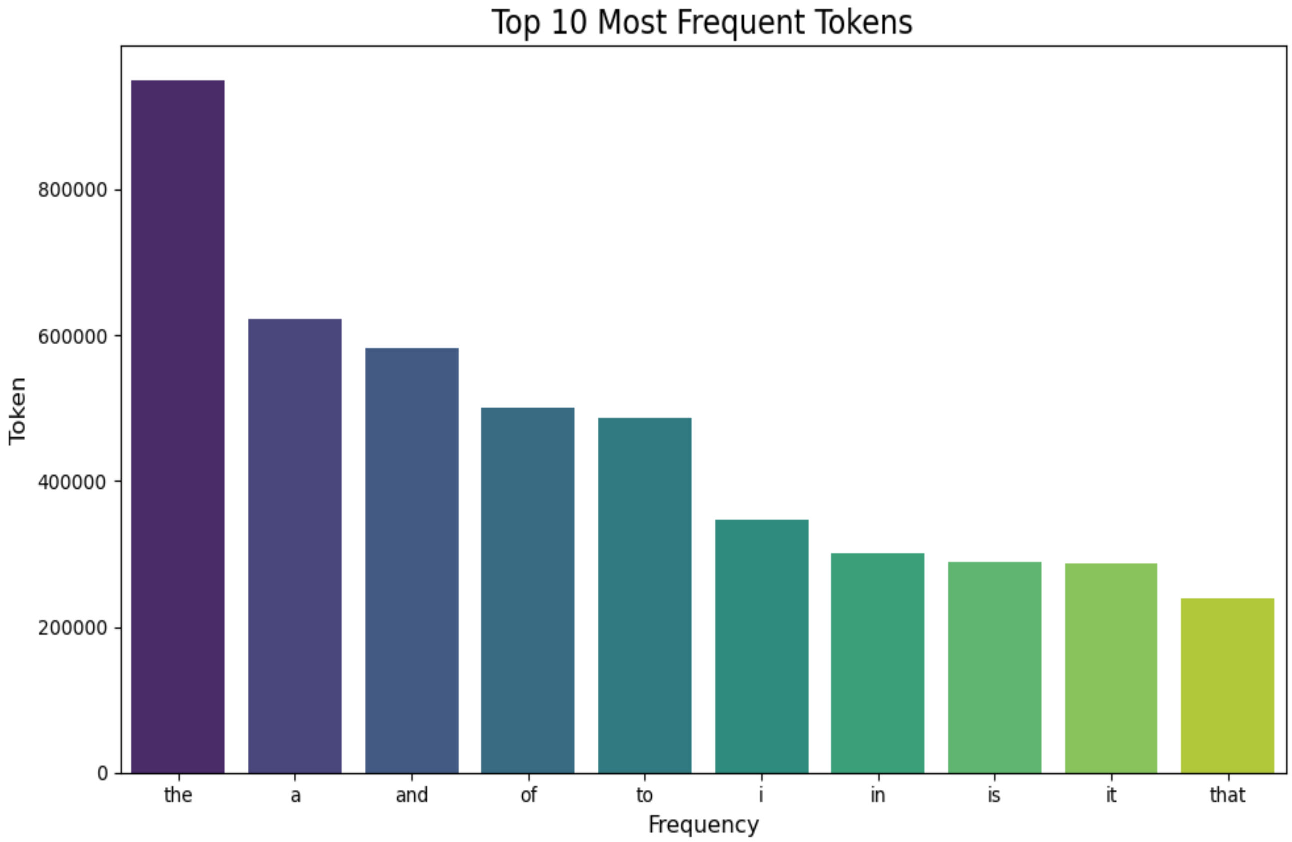
Top 10 Most Frequent Tokens (After Cleaning)

Token	**Frequency**
the	949429
a	621913

and	582635
of	501417
to	487114
i	346223
in	300774
is	289234
it	286797
that	239429

3. Visualizations:

Bar Plot: Top 10 most frequent tokens



Most Common Lemmatized tokens